

LAB 3

Thực hành Tổng hợp

Fullstack Integration: NextJS + Express

Chủ đề	Kết nối Frontend NextJS với Backend Express để quản lý bài viết / bình luận
Thời lượng	5 tiết
Yêu cầu	Node.js 18+, npm, VSCode, kiến thức JavaScript cơ bản

Tiết 1: Thiết lập môi trường & Xử lý CORS

Mục tiêu: Dựng xong 2 server chạy song song, giao tiếp được với nhau mà không bị lỗi CORS.

Cấu trúc dự án

TERMINAL — KHỞI TẠO PROJECT

```
# Tạo thư mục gốc
mkdir fullstack-blog && cd fullstack-blog

# Backend
mkdir backend && cd backend
npm init -y
npm install express cors dotenv

# Frontend (mở tab terminal mới)
cd ..
npx create-next-app@latest frontend --typescript --tailwind --app
```

Cấu trúc thư mục sau khi khởi tạo:

```
fullstack-blog/
├── backend/
│   ├── server.js
│   └── package.json
└── frontend/
    ├── app/
    ├── lib/
    └── package.json
```

Bước 1 — Dựng Backend Express cơ bản

BACKEND/SERVER.JS

```
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({
  origin: 'http://localhost:3000', // chỉ cho phép NextJS
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type']
}));
app.use(express.json());
```

```
let posts = [
  { id: 1, title: 'Bài viết đầu tiên', content: 'Nội dung bài 1', author: 'Admin' },
  { id: 2, title: 'Hướng dẫn NextJS', content: 'Nội dung bài 2', author: 'Admin' },
];

app.get('/api/posts', (req, res) => {
  res.json(posts);
});

app.listen(5000, () => {
  console.log('Backend chạy tại port :5000')
});
```

💡 Chạy backend: `node server.js` — giữ terminal này mở suốt buổi thực hành.

Bước 2 — Cố tình tái hiện lỗi CORS

Thực hiện theo thứ tự để hiểu bản chất lỗi CORS:

- Tạm thời xóa dòng `app.use(cors(...))` trong `server.js`, restart backend.
- Từ NextJS gọi `fetch('http://localhost:5000/api/posts')`, mở DevTools → Console → quan sát thông báo lỗi.
- Chụp màn hình lỗi. Ghi nhận: trình duyệt chặn ở phía nào (client hay server)?
- Khôi phục lại middleware `cors()`, reload trang — lỗi biến mất.

⚠️ Lỗi CORS xảy ra ở trình duyệt (browser), KHÔNG phải ở server hay terminal. Header `Access-Control-Allow-Origin` phải do server gửi về.

Bước 3 — Cấu hình Proxy trong NextJS (cách thay thế)

NEXT.CONFIG.TS

```
const nextConfig = {
  async rewrites() {
    return [{
      source: '/api/:path*',
      destination: 'http://localhost:5000/api/:path*'
    }];
  }
};

export default nextConfig;
```

💡 Với proxy, frontend gọi `/api/posts` (cùng origin) → NextJS tự chuyển tiếp sang backend. Không cần `cors()` trên Express nữa. Đây là cách phổ biến trong môi trường production.

Tiết 2: Kết nối Form → API Backend

Mục tiêu: Tạo form tạo bài viết mới ở Frontend, gửi dữ liệu lên POST `/api/posts` bằng `fetch` và `axios`.

Bước 4 — Thêm route POST trên Backend

BACKEND/SERVER.JS — THÊM VÀO CUỐI FILE

```
app.post('/api/posts', (req, res) => {
  const { title, content, author } = req.body;
```

```
// Validation đơn giản
if (!title || !content || !author)
  return res.status(400).json({ error: 'Thiếu dữ liệu' });

const newPost = {
  id: Date.now(),
  title,
  content,
  author,
  createdAt: new Date().toISOString()
};
posts.push(newPost);
res.status(201).json(newPost);
});
```

Bước 5 — Form với fetch thuần

FRONTEND/APP/POSTS/PAGE.TSX

```
'use client';
import { useState, useEffect } from 'react';

export default function PostsPage() {
  const [posts, setPosts] = useState([]);
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  const [author, setAuthor] = useState('');

  const fetchPosts = async () => {
    const res = await fetch('http://localhost:5000/api/posts');
    const data = await res.json();
    setPosts(data);
  };

  useEffect(() => { fetchPosts(); }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await fetch('http://localhost:5000/api/posts', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, content, author }),
    });
    if (res.ok) {
      setTitle(''); setContent(''); setAuthor('');
      fetchPosts(); // refresh danh sách
    }
  };

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input value={title} onChange={e => setTitle(e.target.value)}
placeholder='Tiêu đề' />
        <textarea value={content} onChange={e => setContent(e.target.value)}
placeholder='Nội dung' />
        <input value={author} onChange={e => setAuthor(e.target.value)}
placeholder='Tác giả' />
        <button type='submit'>Đăng bài</button>
      </form>
      {posts.map(p => <div key={p.id}><h3>{p.title}</h3><p>{p.author} -
{p.content}</p></div>)}
    </div>
  );
}
```

Bước 6 — Nâng cấp dùng axios

CÀI ĐẶT AXIOS

```
npm install axios
```

FRONTEND/LIB/API.TS — FILE CẤU HÌNH TRUNG TÂM

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000',
  headers: { 'Content-Type': 'application/json' },
});

export default api;
```

DÙNG TRONG COMPONENT

```
import api from '@/lib/api';

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    await api.post('/api/posts', { title, content, author });
    fetchPosts();
  } catch (err) {
    console.error(err.response?.data?.error);
  }
};
```

💡 Ưu điểm axios: tự parse JSON, xử lý error dễ hơn qua `err.response`, dễ thêm interceptor (auth token, logging) sau này. Nên dùng axios cho dự án thực tế.

Tiết 3: Toast Message & Debug

Mục tiêu: Hiển thị thông báo phản hồi người dùng và biết cách debug lỗi giữa 2 phía.

Bước 7 — Tích hợp Toast với react-hot-toast

CÀI ĐẶT

```
npm install react-hot-toast
```

FRONTEND/APP/LAYOUT.TSX — THÊM TOASTER VÀO ROOT LAYOUT

```
import { Toaster } from 'react-hot-toast';

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        {children}
        <Toaster position='top-right' />
      </body>
    </html>
  );
}
```

CÁC LOẠI TOAST SỬ DỤNG TRONG COMPONENT

```
import toast from 'react-hot-toast';
```

```
// Toast thành công
toast.success('Đăng bài thành công!');

// Toast xoá
toast.success('Đã xoá bài viết', { icon: '🗑️' });

// Toast lỗi
toast.error('Không thể kết nối server!');

// Async toast: loading → success/error tự động
toast.promise(
  api.post('/api/posts', data),
  {
    loading: 'Đang lưu...',
    success: 'Lưu thành công!',
    error: 'Có lỗi xảy ra!',
  }
);
```

☑️ *toast.promise() là cách tốt nhất cho thao tác async: tự hiện loading, rồi chuyển sang success/error khi promise resolve/reject.*

Bước 8 — Debug lỗi hệ thống

Phía Frontend — Browser DevTools

- DevTools → tab Network: xem request đã gửi đi chưa, status code bao nhiêu.
- Tab Headers: kiểm tra request headers, method, payload đã đúng chưa.
- Tab Response: đọc message lỗi server trả về.
- Tab Console: xem lỗi JavaScript, uncaught promise rejection.

Phía Backend — Terminal Express

- Thêm console.log(req.body) đầu mỗi route để xác nhận data nhận được.
- Thêm middleware log tổng quát: `app.use((req, _, next) => { console.log(req.method, req.url); next() })`
- Kiểm tra port đang chạy: `lsof -i :5000` (macOS/Linux) hoặc `netstat -ano | findstr :5000` (Windows).
- Status code: 400 = lỗi từ client gửi sai, 500 = lỗi trong code server.

Bảng lỗi thường gặp

Lỗi / Thông báo	Nguyên nhân & Hướng xử lý
CORS error	Thiếu middleware cors() hoặc sai origin trong allowedOrigins
404 Not Found	Sai URL, sai HTTP method (GET thay vì POST)
400 Bad Request	Body rỗng hoặc thiếu header Content-Type: application/json
500 Internal Error	Bug trong code Express, kiểm tra terminal backend
ERR_CONNECTION_REFUSED	Backend chưa chạy hoặc cấu hình sai port

Tiết 4-5: Bài tập tự làm & Phần nâng cao

Mục tiêu: Hoàn thiện chức năng Xóa bài viết end-to-end và tự động cập nhật danh sách.

Bài tập bắt buộc: Xóa bài viết end-to-end

Phần 1 — Backend: thêm route DELETE

BACKEND/SERVER.JS — THÊM VÀO CUỐI FILE

```
app.delete('/api/posts/:id', (req, res) => {
  const id = Number(req.params.id);
  const index = posts.findIndex(p => p.id === id);

  if (index === -1)
    return res.status(404).json({ error: 'Không tìm thấy bài viết' });

  posts.splice(index, 1);
  res.json({ message: 'Đã xóa thành công' });
});
```

Phần 2 — Frontend: hàm xóa + revalidate

THÊM VÀO COMPONENT POSTSPAGE

```
const handleDelete = async (id: number) => {
  // 1. Xác nhận người dùng
  if (!confirm('Bạn chắc chắn muốn xóa bài viết này?')) return;

  try {
    // 2. Gọi API xóa
    await api.delete(`/api/posts/${id}`);

    // 3. Cập nhật state NGAY (optimistic update)
    // - không cần gọi lại API, UX nhanh hơn
    setPosts(prev => prev.filter(p => p.id !== id));

    // 4. Hiển thị toast thành công
    toast.success('Đã xóa bài viết');
  } catch (err) {
    toast.error('Xóa thất bại, thử lại!');
    // Rollback: gọi lại server để đồng bộ dữ liệu
    fetchPosts();
  }
};
```

Phần 3 — UI: nút Xóa trong danh sách

```
{posts.map(p => (
  <div key={p.id} className='flex justify-between items-center p-3 border rounded mb-2'>
    <div>
      <h3 className='font-bold'>{p.title}</h3>
      <p className='text-sm text-gray-500'>{p.author} · {p.content}</p>
    </div>
    <button
      onClick={() => handleDelete(p.id)}
      className='text-red-500 hover:text-red-700 text-sm font-medium'
    >
      Xóa
    </button>
  </div>
))}
```

☑ Kiểm tra: sau khi nhấn Xóa và xác nhận, bài viết biến mất khỏi danh sách NGAY LẬP TỨC mà không cần F5 trang. Đây là optimistic update.

Checklist tự kiểm tra trước khi nộp bài

- | | |
|--------------------------|---|
| ☐ | Backend chạy tại port 5000, có đủ 3 route: GET / POST / DELETE /api/posts |
| ☐ | Frontend gọi API thành công, không còn lỗi CORS trong Console |

[]	Form đăng bài hoạt động, dữ liệu mới xuất hiện ngay trong danh sách
[]	Nút Xoá hoạt động, danh sách tự cập nhật sau khi xoá (không cần F5)
[]	Toast hiện khi đăng bài thành công, xoá thành công, và khi có lỗi
[]	Không có lỗi đỏ trong Console (cả browser và terminal backend)

Phần Nâng Cao (BONUS)

Hoàn thành bài tập cơ bản trước. Phần nâng cao để đạt điểm xuất sắc và chuẩn bị cho dự án thực tế.

Nâng cao 1 — Chỉnh sửa bài viết (PUT)

Thêm route PUT /api/posts/:id trên backend để cập nhật thông tin bài viết:

```
app.put('/api/posts/:id', (req, res) => {
  const id = Number(req.params.id);
  const index = posts.findIndex(p => p.id === id);

  if (index === -1)
    return res.status(404).json({ error: 'Không tìm thấy' });

  posts[index] = { ...posts[index], ...req.body };
  res.json(posts[index]);
});
```

Trên frontend: thêm nút Sửa ở mỗi bài viết. Khi bấm, hiển thị inline form (hoặc modal) với tiêu đề và nội dung hiện tại. Gửi PUT request khi submit, cập nhật lại danh sách.

Nâng cao 2 — React Query (TanStack Query) ★ Khuyến khích

💡 Đây là pattern chuẩn trong dự án thực tế. Nên ưu tiên hoàn thành mục này.

```
npm install @tanstack/react-query
```

Thay useEffect + fetch thủ công bằng useQuery và useMutation:

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

const queryClient = useQueryClient();

// Fetch danh sách — tự cache & refetch
const { data: posts, isLoading } = useQuery({
  queryKey: ['posts'],
  queryFn: () => api.get('/api/posts').then(r => r.data)
});

// Mutation xóa — tự invalidate cache sau khi thành công
const deleteMutation = useMutation({
  mutationFn: (id) => api.delete(`/api/posts/${id}`),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['posts'] });
    toast.success('Đã xóa bài viết');
  },
  onError: () => toast.error('Xóa thất bại!'),
});
```

Mục tiêu hiểu: caching tự động, invalidateQueries để revalidate, staleTime để kiểm soát tần suất refetch.

Nâng cao 3 — Lưu dữ liệu vào file JSON

Thay mảng posts trong RAM bằng đọc/ghi file data.json dùng fs.promises. Dữ liệu không mất khi restart server:

```
const fs = require('fs').promises;
const PATH = './data.json';

async function readData() {
  const raw = await fs.readFile(PATH, 'utf-8');
  return JSON.parse(raw);
}
```

```
async function writeData(data) {
  await fs.writeFile(PATH, JSON.stringify(data, null, 2));
}

// Dùng trong route:
app.get('/api/posts', async (req, res) => {
  const posts = await readData();
  res.json(posts);
});
```

Nâng cao 4 — Chức năng Bình luận (Comments)

Xây dựng thêm module Comments hoàn chỉnh:

- Backend: thêm route POST `/api/posts/:id/comments` nhận `{ author, content }`, GET `/api/posts/:id/comments`, DELETE `/api/comments/:commentId`
- Frontend: ô nhập bình luận dưới mỗi bài viết
- Hiển thị số lượng bình luận trên từng bài (cập nhật real-time)
- Mở rộng trang `/posts/:id`: hiển thị bài viết đầy đủ + danh sách bình luận, nút xoá bình luận từng item

 *Nâng cao 2 (React Query) được khuyến khích nhất vì đây là pattern chuẩn trong môi trường production, giúp bạn hiểu sâu về data fetching và cache management.*